Implementação Back-End — Aprova.AI

GWBR Technologies · Documento gerado a partir de `docs/IMPLEMENTACAO-BACKEND.md` em 30/10/2026

Guia para a equipe de API, Workers e dados. Reúne tudo o que é preciso saber para implementar o backend: decisões, pilha, estrutura de `backend/`, modelo de dados, idempotência, o comando `SubmitEssay` com Premium e cota, outbox transacional, filas BullMQ, Workers e integração com a IA, projeções, API HTTP, autenticação, observabilidade, segurança, testes e ordem de implementação. Deriva de `docs/ARQUITETURA.md`; em caso de divergência, o documento de arquitetura prevalece.

Sumário

1. Base comum a todas as equipes
2. Ponto de partida e responsabilidades do back-end
3. Pilha tecnológica proposta
4. Estrutura de `backend/` e regras de dependência
5. Modelo de dados
6. Idempotência de comandos
7. O comando `SubmitEssay`: Premium, cota e gravação
8. Outbox transacional e relay
9. Filas BullMQ e Workers
10. Worker de correção de redação e integração com a IA
11. Projeções e lado de leitura
12. API HTTP
13. Autenticação e identidade
14. Plano Premium: `EntitlementPort`
15. Observabilidade
16. Segurança e LGPD
17. Testes
18. Variáveis de ambiente
19. Ordem de implementação e critérios de aceite
20. Checklist de entrega e pendências

1. Base comum a todas as equipes

Este capítulo é idêntico nos três guias de implementação (Front-End, Back-End e DevOps). Ele existe para que nenhuma equipe dependa de ler o guia da outra para entender o produto, as decisões, o recurso Premium, os contratos e os custos.

1.1 O produto e o objetivo da arquitetura

O Aprova.AI é um aplicativo para Android e iOS, feito com Expo e TypeScript, para estudo para o Enem, a Fuvest e outros vestibulares brasileiros, com correção e apoio por IA (GPT 6.1 - Terra). O objetivo técnico é duplo: o app nunca trava e nenhuma atividade em segundo plano se perde.

Para isso o sistema separa o que o aluno faz na tela (comandos, que mudam estado e respondem na hora) do que o sistema processa nos bastidores (eventos, fila e Workers, onde mora a IA) e do que o

aluno consulta (modelos de leitura, eventualmente consistentes). Quando o aluno envia uma redação, a API responde `202 Accepted` em milissegundos, o app mostra “Redação recebida com sucesso!” e um Worker corrige a redação sem risco de timeout da API.

1.2 Estado de partida verificado em 30/10/2026

Item	Valor verificado
Expo, React Native, React	`expo ~57.0.24`, `react-native 0.86.3`, `react 19.2.3`
TypeScript	`~6.0.3`, modo estrito (`tsconfig.json` estende `expo/tsconfig.base`)
Entrada do app	`index.ts` na raiz (`registerRootComponent`), que importa `src/app/App.tsx` (uma tela vazia)
Scripts do `package.json`	Somente `start`, `android`, `ios` e `web`. **Não existem** scripts de lint, verificação de tipos nem testes
Backend e infraestrutura	Não há código. Existem somente `README.md` de camada em `backend/src/*`, `contracts/`, `src/features/` e `src/core/`
Repositório	`GWBR-Technologies/Aprova.AI`, **público**, branches `master` e `development`
`.gitignore`	Cobre `.env*.local`, mas **não** `.env`; deve ser corrigido antes do primeiro uso de variáveis de ambiente
Dependências de runtime hoje	Apenas `expo`, `expo-status-bar`, `react` e `react-native`

Duas observações que importam para todos. Primeiro, as regras do projeto citam `npm run lint` como o comando de qualidade, mas esse script não existe no `package.json`; a proposta P8 (§1.5) cria o script, e até lá nenhuma equipe pode afirmar que o lint passou. Segundo, como o repositório é público, tudo o que for versionado, inclusive este guia e os logs da integração contínua, é visível a qualquer pessoa: segredos, chaves e dados de alunos nunca entram nele.

1.3 Regras do projeto que valem para todos

- Baseie decisões no código e na configuração existentes, não em suposições sobre o produto. Declare uma suposição somente quando ela afetar o resultado; se a escolha alterar escopo, dados de medição ou risco para o usuário, peça direção.
- Mudanças enxutas: altere só os arquivos necessários, não refatore código vizinho, remova apenas o que a própria mudança tornou obsoleto e não crie abstração para um único uso.
- Não adicione dependência sem aprovação explícita. As propostas do §1.5 estão listadas justamente para serem aprovadas uma a uma.
- Não invente dados de produto: banco de questões, notas, correções, rubricas, preços e textos de marketing vêm do produto. Quando faltar o requisito, pare e peça.
- Siga os padrões existentes (React Native, componentes temáticos, arquivos específicos de plataforma) e os tokens de `src/constants/theme.ts`, que nasce no primeiro uso.
- Comentários explicam decisões, restrições e comportamento não óbvio; não há comentários TODO.
- Informe sempre os comandos executados, os resultados, as limitações e as plataformas que não puderam ser verificadas. Não afirme que algo funciona sem evidência da validação correspondente.
- Não leia, imprima, altere nem versione segredos, `.env` ou `.npmrc`. Não use comandos destrutivos (`rm -rf`, `git clean`, `git reset --hard`, push forçado).

- Commits seguem o padrão `(tipo): Frase no passado`, por exemplo `(docs): Adicionado README.md completo e estruturado desta fase inicial`, com um commit por assunto. Mensagens de commit e descrições de PR não mencionam ferramentas de assistência nem coautoria.
- Qualquer mudança de decisão ou de estrutura atualiza `docs/ARQUITETURA.md`, o `README.md`, os guias e os PDFs, na ordem do §13 do documento de arquitetura.

1.4 Decisões já tomadas

Id	Decisão
D1	Arquitetura geral: Event-Driven Architecture + CQRS.
D2	Fila: BullMQ + Redis, atrás de uma porta única (`QueuePort`); SQS+SNS e RabbitMQ são alternativas.
D3	Comandos respondem de imediato (`202 Accepted`); o trabalho pesado roda em Workers.
D4	O app mantém um outbox local e envia cada comando com chave de idempotência (`commandId`).
D5	O servidor grava estado e mensagem de saída na mesma transação (outbox transacional) e um relay publica na fila.
D6	A IA só é chamada por Workers, atrás de `AiPort` .
D7	Repositório único: app Expo na raiz, `backend/` e `contracts/` , sem workspaces nesta fase.
D8	O lado de leitura é eventualmente consistente; a interface mostra estados explícitos.
D9	Correção de redação por IA é exclusiva do plano `PREMIUM` , com 4 correções por mês (§1.6).
D10	Todo recurso precisa de opção gratuita; custo só com aprovação explícita, exceto a IA (§1.8).
D11	Hospedagem inicial na Oracle Cloud Always Free com Docker Compose; migração planejada para o Render (pago).
D12	Lançamento em Android e iOS; as contas de desenvolvedor da Apple e do Google já existem e as licenças ficam fora do orçamento.
D13	O domínio existente (Hostinger) serve a API por um subdomínio `api.` apontado para o servidor.

1.5 Propostas que aguardam aprovação

Estas propostas são o caminho gratuito que os guias assumem. Cada uma só vira decisão depois de aprovada; até lá, ninguém instala o pacote nem cria o recurso. Todos os pacotes citados são de código aberto (licenças verificadas no registro do npm em 30/10/2026).

Id	Proposta	Custo
P1	Autenticação: login com Google e com Apple (OIDC); a API valida o token do provedor e emite JWT de acesso curto e refresh token rotativo. Sem senha e sem e-mail transacional.	US$ 0
P2	Banco: PostgreSQL 18 (suporte até 14/11/2030) em uma instância, com os esquemas `write` e `read` separados.	US$ 0
P3	Backend: Node.js 24 LTS executando TypeScript nativamente (sem etapa de build), Fastify 5, Drizzle ORM com `pg` , BullMQ 6 com `ioredis` , `zod` 4 (só no backend), `pino` , `prom-client` e o SDK `openai` .	US$ 0 (exceto a IA)
P4	Canal de eventos para o app: consulta periódica com intervalo crescente, com push do Expo como aviso sem dados pessoais.	US$ 0

Id	Proposta	Custo
P5	App: Expo Router, TanStack Query, `expo-sqlite`, `expo-secure-store`, `expo-notifications`, `@react-native-community/netinfo`, `expo-crypto`, `@react-native-google-signin/google-signin` e `expo-apple-authentication`.	US$ 0
P6	`contracts/` continua só com tipos; o backend valida com `zod` e garante a compatibilidade com os tipos em tempo de compilação. D7 permanece como está.	US$ 0
P7	Testes: Vitest no backend; `jest-expo` e `@testing-library/react-native` no app.	US$ 0
P8	Scripts `lint` e `typecheck` na raiz e em `backend/` (ESLint com `eslint-config-expo` no app).	US$ 0
P9	Observabilidade: Sentry (plano gratuito) para erros e Grafana Cloud (plano gratuito) para métricas e logs.	US$ 0
P10	Entrega: GitHub Actions (repositório público), GitHub Container Registry e EAS Build no plano gratuito.	US$ 0

A coluna de custo vale enquanto o uso ficar dentro dos limites gratuitos do §1.8. Se algum limite for ultrapassado, a equipe responsável avisa antes, com o valor estimado.

1.6 Recurso Premium e cota de correção de redação (D9)

Esta é uma regra de produto que atravessa as três equipes.

Regra. Apenas usuários com plano `PREMIUM` ativo podem enviar redação para correção por IA, e cada um tem direito a 4 correções por mês. A regra é aplicada pelo servidor no comando `SubmitEssay`; o app apenas reflete o resultado.

Premissas a confirmar com o produto (nenhuma foi definida na conversa de arquitetura):

- A janela da cota é o mês corrido (do dia 1 às 23h59 do último dia) no fuso `America/Sao_Paulo`, e não 30 dias móveis.
- O plano que não é Premium chama-se `FREE` nos contratos; o nome de exibição é decisão do produto.
- Uma correção já aceita é concluída mesmo que o usuário perca o Premium durante o processamento; novos envios passam a ser rejeitados.
- A cota consumida pertence ao mês em que a redação foi aceita, mesmo que o Worker termine no mês seguinte.

Como a cota é consumida. A reserva e a gravação da redação ocorrem na mesma transação do comando. O servidor mantém um contador por usuário e por mês e o incrementa com um `UPDATE` condicional que só tem efeito se o valor atual for menor que 4. Se nenhuma linha for afetada, a cota acabou. Isso é atômico mesmo com envios simultâneos em dois aparelhos.

Situação	Resultado	Cota
Usuário sem plano Premium envia redação	`403` e `PREMIUM_REQUIRED`; o app mostra a oferta do Premium. Nada é gravado.	Não consome
Premium com a cota esgotada	`403` e `ESSAY_QUOTA_EXCEEDED` com `limit`, `used` e `resetsAt`. Nada é gravado.	Não consome
Premium com cota disponível	`202`; a redação e a mensagem de saída são gravadas.	Consome 1

Situação	Resultado	Cota
Mesmo `commandId` reenviado (rede ruim)	`202` idêntico ao primeiro.	Não consome de novo
Correção termina em falha permanente (`EssayCorrectionFailed`)	O app mostra a falha e a ação de reenviar.	Devolve 1, na mesma transação que grava a falha
Falha transitória que acaba corrigindo após novas tentativas	`EssayCorrected`.	Mantém o 1 consumido
Virada de mês com a correção em andamento	A correção conclui normalmente.	Fica no mês da submissão

O que a interface precisa mostrar. Para o usuário Premium, quantas correções restam no mês e quando renovam (`used`, `limit` e `resetsAt` vêm da consulta de direitos, §1.7). Para quem não é Premium, a oferta do Premium no ponto de envio da redação; o texto e o preço dependem do produto e não são inventados aqui. Em ambos os casos a informação vem do servidor, nunca de contagem local.

Testes exigidos para esta regra (back-end, com apoio do front-end nas telas): usuário sem Premium; Premium com 0, 3 e 4 usos; duas submissões simultâneas com 3 usos (apenas uma passa); reenvio do mesmo `commandId`; falha permanente devolvendo a cota; virada de mês em `America/Sao_Paulo` (por exemplo 31/10 às 23h59 e 01/11 às 00h00 no horário de Brasília); e perda do Premium durante o processamento.

Em aberto (§1.8 e §14 do documento de arquitetura). Como o usuário se torna Premium (assinatura, cobrança, preço, período de teste) ainda não foi decidido. A arquitetura isola isso atrás da `EntitlementPort`: o backend só pergunta “qual é o plano deste usuário agora?”. Enquanto a fonte real não existir, nenhuma equipe deve criar um mecanismo de concessão de Premium por conta própria.

1.7 Contratos entre o app e o backend

Todos os contratos moram em `contracts/` como **tipos** (sem lógica e sem código executável) e são a única fonte de verdade do formato. Um contrato novo só entra com aprovação.

Comandos. `POST /v1/commands/<nome-em-kebab-case>` com o corpo abaixo. O sucesso é sempre `202`.

```
// contracts/commands/command-envelope.ts (somente tipos)
export type CommandEnvelope<TPayload> = {
  commandId: string; // UUID v4 gerado no app; é a chave de idempotência
  issuedAt: string;  // ISO 8601 em UTC
  payload: TPayload;
};

export type CommandAccepted = {
  commandId: string;
  status: 'accepted';
  receivedAt: string; // ISO 8601 em UTC
};

// contracts/commands/submit-essay.command.ts
export type SubmitEssayPayload = {
  essayId: string; // UUID gerado no app
  text: string;
```

```
  // Tema e limites de tamanho do texto são definidos com o produto (§14).
};
```

Comandos iniciais (§5 do documento de arquitetura): `AnswerQuestion`, `RecordRoutineProgress`, `SubmitEssay` e `CompleteMockExam`. Os campos de negócio de cada payload (identificador da questão, alternativa, tempo, duração e assim por diante) são definidos com o produto antes da implementação; nenhuma equipe os inventa.

Consultas. `GET /v1/...`; toda resposta traz `asOf` (ISO 8601 em UTC) com o instante da última projeção aplicada.

Consulta	Rota	Resposta (resumo)
`GetEssay`	`GET /v1/essays/{essayId}`	`status` (`processing`, `completed`, `failed`), `result` quando concluída, `failure` quando falhou, `asOf`; `404` enquanto a projeção ainda não foi aplicada
`GetEssayList` (proposta, precisa de aprovação)	`GET /v1/me/essays`	Redações do usuário com `status` e data, paginadas, `asOf`
`GetEntitlements`	`GET /v1/me/entitlements`	`plan` (`FREE` ou `PREMIUM`) e `essayQuota` (`limit`, `used`, `resetsAt`) ou `null` quando não há direito, `asOf`
`GetPerformanceSummary`	`GET /v1/me/performance`	Estatísticas de desempenho do read store, `asOf`
`GetMockExamHistory`	`GET /v1/me/mock-exams`	Histórico de simulados, `asOf`
`GetMockExamResult`	`GET /v1/mock-exams/{mockExamId}`	`status` e resultado consolidado, `asOf`

O formato do `result` da redação depende da rubrica (em aberto) e não é definido aqui. O estado `received` do app é do **outbox local** (a API respondeu `202`); no servidor, uma redação aceita já está `processing`, e logo depois do `202` a consulta pode devolver `404` porque a projeção é assíncrona. O app trata esse `404` transitório como “em processamento”.

Erros. Toda resposta de erro tem o mesmo corpo, que nunca expõe detalhes internos:

```
export type ApiError = {
  code: ErrorCode;
  message: string;        // texto compreensível para o aluno
  correlationId: string;  // igual ao commandId quando houver
  details?: Record<string, unknown>;
};
```

Status	`code`	Quando	O app deve
400	`VALIDATION_FAILED`	Corpo, parâmetro ou cabeçalho inválido	Marcar o comando como falho e mostrar a correção necessária
401	`UNAUTHENTICATED`	Token ausente, inválido ou expirado	Renovar o token; se falhar, pedir novo login

Status	code	Quando	O app deve
403	`PREMIUM_REQUIRED`	Plano sem direito à correção	Mostrar a oferta do Premium; não reenviar
403	`ESSAY_QUOTA_EXCEEDED`	Cota mensal esgotada (`limit` , `used` , `resetsAt`)	Informar quando renova; não reenviar
409	`COMMAND_ID_CONFLICT`	Mesmo `commandId` com payload diferente	Tratar como bug; registrar e não reenviar
429	`RATE_LIMITED`	Excesso de requisições	Reenviar depois de `Retry-After`
500	`INTERNAL_ERROR`	Falha inesperada	Reenviar com intervalo crescente
503	`SERVICE_UNAVAILABLE`	Dependência fora do ar	Reenviar com intervalo crescente

Regra de reenvio do outbox: reenviar erros de rede, `408` , `429` e `5xx` ; nunca reenviar os demais `4xx` , que são permanentes e terminam o comando como `failed` .

Autenticação. `POST /v1/auth/login` (`{ provider, idToken }` , com `provider` igual a `google` ou `apple`), `POST /v1/auth/refresh` e `POST /v1/auth/logout` são endpoints HTTP síncronos, fora do modelo de comandos, porque devolvem tokens e não `202` . O login devolve `{ accessToken, accessTokenExpiresAt, refreshToken }` . Dependem da proposta P1.

Comando técnico proposto. `RegisterPushToken` (idempotente por token) só existe se a proposta P4 for aprovada; não faz parte do conjunto inicial.

Aviso por push. O push do Expo é só um aviso para o app consultar de novo. O corpo é `{ type, aggregateId, correlationId }` , com `type` igual a `EssayCorrected` , `EssayCorrectionFailed` ou `MockExamProcessed` . Nunca carrega texto de redação, nota ou dado pessoal.

Evolução. As rotas são versionadas por prefixo (`/v1`). Mudança compatível (campo opcional novo) não muda a versão; mudança incompatível cria `/v2` e mantém `/v1` até o último app que a usa sair de circulação.

1.8 Custos e recursos gratuitos (D10)

Regra do projeto: toda decisão deve ter opção gratuita, exceto a IA. A tabela abaixo lista cada recurso, o limite gratuito consultado em 30/10/2026 nas páginas oficiais, o que acontece ao passar dele e o custo por 1.000 usuários quando faz sentido. Itens marcados com † vêm de conhecimento anterior e não foram reverificados nesta data.

Recurso	Limite gratuito	Custo ao ultrapassar	Por 1.000 usuários
Oracle Cloud Always Free (D11)	VM Ampere A1 (2 OCPU e 12 GB no total, conforme a página consultada), 2 VMs AMD micro (1 GB cada), 200 GB de disco, 10 TB/mês de saída	Passar a Pay As You Go; o que fica dentro do Always Free segue sem cobrança	US$ 0 (carga a validar em teste)
GitHub Actions e repositório	Uso gratuito para repositório público	Repositório privado tem créditos incluídos limitados; consultar antes de privatizar	US$ 0
GitHub Container Registry †	Gratuito para pacotes públicos	Não verificado	US$ 0

Recurso	Limite gratuito	Custo ao ultrapassar	Por 1.000 usuários
EAS Build (plano Free)	15 builds Android e 15 iOS por mês, fila de baixa prioridade	Starter US$ 19/mês (US$ 45 de crédito de build); Production US$ 199/mês (US$ 225)	US$ 0
EAS Update (opcional)	1.000 usuários ativos por mês	Starter inclui 3.000 e Production 50.000; acima, preço por uso (valor não publicado na página)	US$ 0 até 1.000 usuários ativos
Sentry (plano Developer)	5 mil erros, 1 usuário, 30 dias de histórico	Team: US$ 29/mês (US$ 26 no plano anual)	US$ 0
Grafana Cloud (plano Free)	10 mil séries de métricas, 50 GB de logs, 50 GB de traces, 3 usuários, 14 dias de retenção	Pro a partir de US$ 19/mês mais uso	US$ 0
HTTPS (Let's Encrypt pelo Caddy) †	Gratuito	—	US$ 0
Login com Google e com Apple †	Gratuito (a Apple exige a conta de desenvolvedor, que já existe)	—	US$ 0
Push do Expo	A documentação consultada não informa preço nem limites	Confirmar antes de depender em volume	A confirmar
Credenciais de push do Android (Firebase) †	Projeto Firebase gratuito	—	US$ 0
Domínio e DNS (D13)	Domínio já existente na Hostinger	—	US$ 0
Contas Apple e Google Play (D12)	Já existem; licenças fora do orçamento	—	Fora do escopo
Render (rota de migração)	Redis gratuito de 25 MB sem persistência em disco: incompatível com o BullMQ	Web US$ 7/mês, Postgres US$ 6/mês (1 GB), Key Value US$ 10/mês (256 MB), total US$ 23/mês; Worker separado não verificado	≈ US$ 0,023 por usuário/mês (US$ 23 a cada 1.000)

Custo da IA (GPT 6.1 - Terra), único custo que cresce por usuário. Preços por 1 milhão de tokens na página oficial de preços da OpenAI (modelo `gpt-6.1-terra`, camada padrão): entrada US$ 1,00; entrada em cache US$ 0,05; saída US$ 5,00; gravação em cache de 30 minutos US$ 1,25; de 12 horas US$ 2,50. A mesma página mostra uma segunda linha para o modelo com metade desses valores, que parece ser a camada flex ou batch (o rótulo não está explícito, por isso não é usada nas contas).

As quantidades de tokens abaixo são **estimativas de arquitetura, não medições**: cerca de 3.500 tokens de entrada por correção (texto da redação mais prompt e critérios, dos quais cerca de 1.500 formam um prefixo estável) e 1.000 de saída. A tabela de custo por 1.000 assinantes Premium assume o teto de uso (todos usam as 4 correções do mês, ou seja, 4.000 correções):

Cenário	Custo por correção	Custo por mês a cada 1.000 assinantes Premium
Sem cache (3.500 de entrada e 1.000 de saída)	US$ 0,0085	**US$ 34,00**
Prefixo de 1.500 tokens servido do cache em todas as chamadas	US$ 0,0071	US$ 28,30
Saída de 2.000 tokens, sem cache	US$ 0,0135	US$ 54,00

O custo por assinante fica em no máximo US$ 0,034 por mês no cenário base. Para estimar o custo por mil usuários totais, multiplique pela fração de assinantes: com 10% de Premium, US$ 3,40 a cada 1.000 usuários; com 25%, US$ 8,50. O cache só compensa se o prefixo for reaproveitado: escrever custa 25% a mais que a entrada comum e cada leitura economiza cerca de 95%, então basta uma leitura a cada quatro escritas para valer a pena. Todos os números serão substituídos pelo consumo real registrado pelo Worker (tabela `ai_usage`, guia do Back-End) logo nas primeiras semanas.

Custos que dependem do Premium e ainda não foram decididos. Se o Premium for vendido pelas lojas, as comissões consultadas hoje são: Apple 15% no Small Business Program (receita de até US$ 1 milhão no ano anterior); Google Play 15% sobre assinaturas nos mercados que não estão entre os de estrutura nova (a página lista Austrália, Espaço Econômico Europeu, Japão, Reino Unido e EUA com 10% mais 5% de cobrança). Se for usado o RevenueCat para gerir assinaturas, a página informa que não há cobrança até US$ 2.500 de receita mensal rastreada e 1% acima disso (a página não esclarece se o 1% incide só sobre o excedente). Se o Premium precisa ou não usar a cobrança das lojas depende das diretrizes de cada loja e deve ser confirmado antes de implementar. Nada disso está aprovado: o item segue em aberto (§14 do documento de arquitetura).

1.9 Glossário

Termo	Significado
Comando	Intenção do aluno de mudar o estado; verbo no imperativo (`SubmitEssay`). Responde `202`, nunca o resultado.
Consulta	Leitura de um modelo de leitura; nunca muda estado; devolve `asOf`.
Evento	Fato já ocorrido e imutável, nomeado no passado (`EssaySubmitted`).
`commandId`	UUID gerado no app; chave de idempotência e `correlationId` de toda a cadeia.
Outbox local	Fila de comandos pendentes no aparelho, enviada quando houver conexão.
Outbox transacional	Tabela do servidor gravada na mesma transação do estado; um relay publica na fila.
Relay	Processo que lê o outbox transacional e publica no BullMQ.
Worker	Processo sem servidor HTTP que consome uma fila (correção de redação, simulados, projeções).
Projeção (read model)	Tabela desnormalizada atualizada por eventos, usada só por consultas.
`asOf`	Instante da última projeção aplicada; indica o possível atraso do dado.
Dead-letter	Fila de mensagens que esgotaram as tentativas ou falharam de forma permanente.
AOF	Persistência do Redis por log de comandos, exigida pelo BullMQ em produção.
Premium	Plano com direito à correção de redação por IA (4 por mês).

Termo	Significado
MAU	Usuários ativos por mês (base de cobrança do EAS Update).

1.10 Fontes consultadas em 30/10/2026

Preços, limites e versões deste guia vêm das páginas oficiais abaixo. Quem for implementar deve reconferi-las, porque planos e limites mudam.

- Oracle Always Free: `docs.oracle.com/en-us/iaas/Content/FreeTier/freetier_topic-Always_Free_Resources.htm`.
- BullMQ: `docs.bullmq.io/guide/going-to-production`, `.../jobs/job-ids` e `.../retrying-failing-jobs`; tipos do pacote `bullmq` 6.5.2.
- Expo: `expo.dev/pricing` e `docs.expo.dev/push-notifications/overview`.
- Render: `render.com/pricing`. Sentry: `sentry.io/pricing`. Grafana: `grafana.com/pricing`.
- OpenAI: `developers.openai.com/api/docs/pricing` (linha `gpt-6.1-terra`).
- Lojas e assinaturas: `developer.apple.com/app-store/small-business-program`, `support.google.com/googleplay/android-developer/answer/112622` e `revenuecat.com/pricing`.
- Ciclo de vida: `nodejs.org/en/about/previous-releases`, `endoflife.date/postgresql` e `endoflife.date/redis`.
- Versões e licenças de pacotes: registro do npm, consultado em 30/10/2026.

2. Ponto de partida e responsabilidades do back-end

Não existe código de backend. O repositório tem apenas os `README.md` de camada em `backend/src/{domain,application,infrastructure,api,workers,bootstrap}` e `contracts/`. Tudo o que está neste guia nasce incrementalmente, no primeiro uso real.

O backend tem seis responsabilidades:

1. **Aceitar comandos** de forma idempotente e responder `202` em milissegundos, gravando estado e mensagem de saída na mesma transação (D3, D5).
2. **Aplicar a regra do Premium e da cota** no comando `SubmitEssay` (D9, §7).
3. **Publicar mensagens** com um relay do outbox para o BullMQ (§8).
4. **Processar em Workers** a correção de redação (única chamada à IA), os simulados e as projeções (D6).
5. **Servir consultas** a partir de modelos de leitura, sempre com `asOf` (D8).
6. **Não perder nada e não se repetir**: tudo é pelo menos uma vez e idempotente (§6, §9).

3. Pilha tecnológica proposta

Cada item é proposta (P2, P3, P6, P7 e P8, §1.5) e só entra no `backend/package.json` depois de aprovado. Versões e licenças verificadas no npm em 30/10/2026.

Papel	Escolha	Versão atual	Licença
Runtime	Node.js 24 LTS (Krypton); as linhas 22 e 26 também estão em LTS	—	MIT
Linguagem	TypeScript na mesma versão do app (`~6.0.3`)	6.0.3	Apache-2.0

Papel	Escolha	Versão atual	Licença
HTTP	`fastify`	5.12.7	MIT
Banco	PostgreSQL 18 (suporte até 14/11/2030)	18.6	PostgreSQL
Acesso a dados	`drizzle-orm` e `drizzle-kit` com `pg`	0.45.4, 0.31.11 e 8.23.1	Apache-2.0, MIT e MIT
Fila	`bullmq` com `ioredis`	6.5.2 e 6.0.0	MIT
Redis	Redis 8.2 (segurança até 01/09/2030), com AOF	8.2.11	RSALv2, SSPLv1 ou AGPLv3
Validação	`zod` (somente no backend)	4.6.7	MIT
Tokens	`jose`	6.2.14	MIT
Logs e métricas	`pino` e `prom-client`	10.4.0 e 15.1.3	MIT e Apache-2.0
Segurança HTTP	`@fastify/helmet`, `@fastify/rate-limit`, `@fastify/cors`	13.2.0, 11.2.1 e 11.3.1	MIT
IA	SDK `openai`	7.46.0	Apache-2.0
Erros	`@sentry/node`	11.10.0	MIT
Testes	`vitest`	5.0.5	MIT
Lint	`eslint` com `typescript-eslint`	10.14.0	MIT

Observações sobre a escolha:

- **Sem etapa de build.** O Node 24 executa TypeScript diretamente (remove os tipos, sem checar). A checagem de tipos é o `tsc --noEmit`. Validado em 30/10/2026 numa cópia mínima (Node 22.23.4 e TypeScript 6.0.3): o `tsc` passou com alias para `contracts/` e o Node executou o `.ts` sem aviso. Isso elimina `tsx`, `tsup` e `dist/`. O ponto precisa ser reconfirmado no Node 24 no marco B0.
- **`ioredis` é dependência par do BullMQ**, não vem junto: instale-o explicitamente.
- **O BullMQ 6.5.2 inclui um backend opcional em PostgreSQL** (`PostgresQueueBackend`, com `FOR UPDATE SKIP LOCKED` e `LISTEN/NOTIFY`). A decisão do projeto é BullMQ com Redis (D2) e este guia a segue; o backend em Postgres é uma alternativa a avaliar no futuro (eliminaria o Redis da VM), mas a página do pacote ainda se descreve como baseada em Redis e a maturidade não foi verificada.
- **O BullMQ 6.5.2 não tem opção nativa de dead-letter** (nenhum `deadLetterQueue` nas definições de tipos). A cópia para `dead-letter` é um passo explícito do Worker, como já define o §6 da arquitetura.
- **Drizzle** foi escolhido por declarar o esquema em TypeScript e gerar migrações SQL versionadas. Se a equipe preferir outro acesso a dados, o contrato é o SQL do §5, e a mudança é uma nova proposta.
- **`typescript-eslint`** é a única dependência de lint adicional do backend.

4. Estrutura de `backend/` e regras de dependência

```
backend/
├── package.json            # "type": "module"; scripts do §4.2
├── tsconfig.json           # §4.3
```

```
├── drizzle.config.ts
├── migrations/             # SQL gerado pelo drizzle-kit, versionado
└── src/
    ├── domain/
    │   └── essay/          # essay.ts (agregado), quota-period.ts (função pura)
    ├── application/
    │   ├── commands/       # submit-essay.handler.ts, answer-question.handler.ts, ...
    │   ├── queries/        # get-essay.handler.ts, get-entitlements.handler.ts, ...
    │   ├── events/         # essay-view.projector.ts, performance.projector.ts, ...
    │   └── ports/          # ai.port.ts, queue.port.ts, entitlement.port.ts, ...
    ├── infrastructure/
    │   ├── queue/          # bullmq.queue.adapter.ts, connections.ts
    │   ├── persistence/    # schema.ts (Drizzle), *.repository.ts, outbox.repository.ts
    │   └── ai/             # openai.ai.adapter.ts, prompts/essay-correction.v1.ts
    ├── api/                # server.ts, routes/commands/*.ts, routes/queries/*.ts, errors.ts
    ├── workers/
    │   ├── essay-correction/  # essay-correction.worker.ts
    │   ├── mock-exam-processing/
    │   ├── projections/
    │   ├── notifications/     # envia push (exige P4 aprovada)
    │   └── relay/             # outbox-relay.ts
    └── bootstrap/          # api.ts, worker.ts, relay.ts, migrate.ts (composition roots)
```

4.1 Regras de dependência (resumo do §10.3 da arquitetura)

Camada	Pode importar	Não pode importar
`domain`	Nada além de si	Todo o restante
`application`	`domain`, `contracts`	`infrastructure`, `api`, `workers`, bibliotecas de fila, IA ou banco
`infrastructure`	`application/ports`, `domain`, `contracts`	`api`, `workers`
`api`	`application`, `contracts`	`infrastructure`, `domain`
`workers`	`application`, `contracts`	`infrastructure`, `api`
`bootstrap`	Todas as camadas	—

Só `bootstrap/` conhece adaptadores concretos: é lá que `AiPort` recebe o adaptador da OpenAI e `QueuePort` recebe o BullMQ. Handlers de comando só gravam no write store e no outbox; handlers de consulta só leem o read store (com a exceção documentada de `GetEntitlements`, §14).

4.2 `package.json` e scripts

```
{
  "name": "aprovaai-backend",
  "private": true,
  "type": "module",
  "engines": { "node": ">=24" },
  "scripts": {
    "dev:api": "node --watch src/bootstrap/api.ts",
```

```
    "start:api": "node src/bootstrap/api.ts",
    "start:worker": "node src/bootstrap/worker.ts",
    "start:relay": "node src/bootstrap/relay.ts",
    "typecheck": "tsc --noEmit",
    "lint": "eslint .",
    "test": "vitest run",
    "db:generate": "drizzle-kit generate",
    "db:migrate": "node src/bootstrap/migrate.ts"
  }
}
```

4.3 `tsconfig.json` (validado em 30/10/2026)

```
{
  "compilerOptions": {
    "target": "ES2023",
    "module": "nodenext",
    "moduleResolution": "nodenext",
    "strict": true,
    "noEmit": true,
    "erasableSyntaxOnly": true,
    "verbatimModuleSyntax": true,
    "allowImportingTsExtensions": true,
    "skipLibCheck": true,
    "types": ["node"],
    "paths": { "@contracts/*": ["../contracts/*"] }
  },
  "include": ["src", "../contracts"]
}
```

4.4 Regras para executar TypeScript nativamente

- Todo import relativo termina em `.ts` (`import { x } from './x.ts'`).
- `erasableSyntaxOnly` proíbe `enum`, `namespace` e propriedades de parâmetro (`constructor(private x)`), porque o Node só remove tipos. Teste negativo feito: um `enum` falha no `tsc` e no Node.
- O alias `@contracts/*` só existe para o `tsc`. Como `contracts/` contém apenas tipos, todo import dele é `import type`, que é apagado antes da execução; por isso o alias não precisa existir em tempo de execução e a imagem de produção não precisa copiar `contracts/`.
- Não há aliases para `src/`: use caminhos relativos dentro de `backend/`.
- O `package.json` precisa de `"type": "module"` (sem isso o Node avisa e relê o arquivo).
- O limite da cota (4) **não** está em `contracts/`: é configuração do servidor (`ESSAY_MONTHLY_LIMIT`), e o app lê o valor pela consulta `GetEntitlements`.

5. Modelo de dados

Uma instância PostgreSQL com dois esquemas (P2): `write` (estado e outbox, otimizado para gravação) e `read` (projeções, otimizadas para leitura). Handlers de comando só tocam `write`; handlers de consulta só tocam `read` (exceção em §14). Papéis de banco separados reforçam isso (§16). Migrações

são SQL versionado em `backend/migrations/`, geradas com o `drizzle-kit` e aplicadas por `node src/bootstrap/migrate.ts` antes de iniciar a API.

Os campos de negócio de questões, rotina e simulados **dependem do produto** e não são inventados: as tabelas correspondentes guardam o `payload` em `jsonb` até que os campos finais sejam definidos, quando passam a colunas tipadas por uma nova migração.

5.1 Esquema `write`

```
CREATE SCHEMA IF NOT EXISTS write;
CREATE SCHEMA IF NOT EXISTS read;

CREATE TABLE write.users (
  id           uuid PRIMARY KEY,
  provider     text NOT NULL CHECK (provider IN ('google', 'apple')),
  provider_sub text NOT NULL,
  created_at   timestamptz NOT NULL DEFAULT now(),
  UNIQUE (provider, provider_sub)
);
-- E-mail e nome não são guardados: minimização de dados até haver necessidade (LGPD).

CREATE TABLE write.refresh_tokens (
  id         uuid PRIMARY KEY,
  user_id    uuid NOT NULL REFERENCES write.users (id),
  family_id  uuid NOT NULL,
  token_hash bytea NOT NULL UNIQUE,   -- SHA-256; o token em si nunca é guardado
  expires_at timestamptz NOT NULL,
  revoked_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE write.entitlements (
  user_id     uuid PRIMARY KEY REFERENCES write.users (id),
  plan        text NOT NULL CHECK (plan IN ('FREE', 'PREMIUM')),
  valid_until timestamptz,            -- nulo: sem expiração
  updated_at  timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE write.command_log (
  user_id      uuid NOT NULL,
  command_id   uuid NOT NULL,
  command_type text NOT NULL,
  payload_hash bytea NOT NULL,        -- SHA-256 do payload em JSON canônico
  response     jsonb NOT NULL,        -- o 202 devolvido
  received_at  timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, command_id)
);

CREATE TABLE write.essays (
  id             uuid PRIMARY KEY,    -- essayId gerado no app
  user_id        uuid NOT NULL REFERENCES write.users (id),
  command_id     uuid NOT NULL,
```

```
  text           text NOT NULL,
  status         text NOT NULL CHECK (status IN ('processing', 'completed', 'failed')),
  quota_period   text NOT NULL,       -- 'AAAA-MM' em America/Sao_Paulo
  version        integer NOT NULL DEFAULT 1,   -- aggregateVersion
  result         jsonb,               -- formato definido pela rubrica (em aberto)
  prompt_version text,
  model          text,
  failure_code   text,
  created_at     timestamptz NOT NULL DEFAULT now(),
  updated_at     timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX essays_user_created ON write.essays (user_id, created_at DESC);

CREATE TABLE write.essay_quota (
  user_id uuid NOT NULL,
  period  text NOT NULL,              -- 'AAAA-MM'
  used    integer NOT NULL DEFAULT 0 CHECK (used >= 0),
  PRIMARY KEY (user_id, period)
);

CREATE TABLE write.question_answers (
  id          uuid PRIMARY KEY,
  user_id     uuid NOT NULL,
  command_id  uuid NOT NULL,
  payload     jsonb NOT NULL,         -- campos finais definidos com o produto
  answered_at timestamptz NOT NULL
);

CREATE TABLE write.routine_progress (
  id          uuid PRIMARY KEY,
  user_id     uuid NOT NULL,
  command_id  uuid NOT NULL,
  payload     jsonb NOT NULL,
  recorded_at timestamptz NOT NULL
);

CREATE TABLE write.mock_exams (
  id         uuid PRIMARY KEY,
  user_id    uuid NOT NULL,
  command_id uuid NOT NULL,
  payload    jsonb NOT NULL,
  status     text NOT NULL CHECK (status IN ('processing', 'completed', 'failed')),
  version    integer NOT NULL DEFAULT 1,
  result     jsonb,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE write.outbox (
  id             bigserial PRIMARY KEY,
  event_id       uuid NOT NULL UNIQUE,
  event_type     text NOT NULL,
```

```
  aggregate_id   uuid NOT NULL,
  correlation_id uuid NOT NULL,
  envelope       jsonb NOT NULL,      -- envelope do §5 da arquitetura
  created_at     timestamptz NOT NULL DEFAULT now(),
  published_at   timestamptz,
  attempts       integer NOT NULL DEFAULT 0,
  last_error     text
);
CREATE INDEX outbox_pending ON write.outbox (id) WHERE published_at IS NULL;

CREATE TABLE write.processed_events (
  consumer     text NOT NULL,         -- ex.: 'essay-correction'
  event_id     uuid NOT NULL,
  processed_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (consumer, event_id)
);

CREATE TABLE write.ai_usage (
  id                  bigserial PRIMARY KEY,
  essay_id            uuid NOT NULL,
  attempt             integer NOT NULL,
  model               text NOT NULL,
  prompt_version      text NOT NULL,
  input_tokens        integer NOT NULL,
  cached_input_tokens integer NOT NULL,
  output_tokens       integer NOT NULL,
  cost_usd            numeric(12, 6) NOT NULL,
  created_at          timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE write.push_tokens (       -- exige P4 aprovada
  user_id    uuid NOT NULL REFERENCES write.users (id),
  token      text NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, token)
);
```

Sobre `write.entitlements`: é a tabela que a `EntitlementPort` lê. **Quem a preenche** (webhook de assinatura, painel interno) é decisão em aberto (§1.6); até lá ela só recebe dados de teste em ambiente de desenvolvimento, e nenhuma rota pública concede Premium.

5.2 Esquema `read`

```
CREATE TABLE read.essay_view (
  essay_id          uuid PRIMARY KEY,
  user_id           uuid NOT NULL,
  status            text NOT NULL,
  result            jsonb,
  failure_code      text,
  aggregate_version integer NOT NULL,
  created_at        timestamptz NOT NULL,
```

```
  as_of             timestamptz NOT NULL
);
CREATE INDEX essay_view_user ON read.essay_view (user_id, created_at DESC);

CREATE TABLE read.performance_summary (
  user_id           uuid PRIMARY KEY,
  data              jsonb NOT NULL,   -- campos definidos com o produto
  as_of             timestamptz NOT NULL
);

CREATE TABLE read.mock_exam_view (
  mock_exam_id      uuid PRIMARY KEY,
  user_id           uuid NOT NULL,
  status            text NOT NULL,
  result            jsonb,
  aggregate_version integer NOT NULL,
  created_at        timestamptz NOT NULL,
  as_of             timestamptz NOT NULL
);
CREATE INDEX mock_exam_view_user ON read.mock_exam_view (user_id, created_at DESC);
```

A projeção de `performance_summary` acumula contadores e não pode depender de `aggregateVersion` único; ela usa `write.processed_events` para ser idempotente (§11).

6. Idempotência de comandos

A chave é `(user_id, command_id)`, e a regra é: o mesmo comando enviado duas ou mais vezes produz um único efeito e devolve sempre o mesmo `202`.

Algoritmo, dentro de **uma transação**:

1. Calcular `payload_hash` (SHA-256 do JSON canônico do `payload`, com chaves ordenadas).
2. Tentar reivindicar o comando:

```
INSERT INTO write.command_log (user_id, command_id, command_type, payload_hash, response)
VALUES ($1, $2, $3, $4, '{}'::jsonb)
ON CONFLICT (user_id, command_id) DO NOTHING
RETURNING command_id;
```

3. Se retornou linha, o comando é novo: seguir com as regras do domínio (§7). No fim da mesma transação, `UPDATE write.command_log SET response = $resposta WHERE ...`.
4. Se não retornou linha, o comando já existe. Se outra transação ainda o estiver processando, o `INSERT` espera ela terminar (conflito de índice único) e só então devolve "nada inserido". Ler a linha: se o `payload_hash` for igual, devolver `response` (mesmo `202`, com o cabeçalho `Idempotent-Replayed: true`); se for diferente, responder `409` com `COMMAND_ID_CONFLICT`.
5. Qualquer erro de regra (`PREMIUM_REQUIRED`, `ESSAY_QUOTA_EXCEEDED`, validação) provoca `ROLLBACK`: a linha do `command_log` some junto, nada é gravado e o mesmo `commandId` pode ser avaliado de novo mais tarde.

Consequência importante: o reenvio de um comando **já aceito** devolve o `202` original **mesmo que, nesse meio-tempo, o usuário tenha perdido o Premium ou esgotado a cota**, porque a

verificação de reenvio vem antes das regras. Isso é o que impede a cota de ser cobrada duas vezes por um reenvio.

7. O comando `SubmitEssay`: Premium, cota e gravação

7.1 Passo a passo

Ordem dentro da transação (as validações de borda — corpo, tamanho e tipos — ocorrem antes, na camada `api`):

1. Reivindicar o `commandId` (§6). Se for reenvio, devolver a resposta guardada e terminar.
2. Consultar o plano pela `EntitlementPort`. Se não for `PREMIUM` ativo: `PREMIUM_REQUIRED` (`403`), com `ROLLBACK`.
3. Reservar a cota do mês (SQL abaixo). Se nenhuma linha for devolvida: `ESSAY_QUOTA_EXCEEDED` (`403`) com `limit`, `used` e `resetsAt`, com `ROLLBACK`.
4. Inserir a redação com `status = 'processing'`, `version = 1` e `quota_period`.
5. Inserir no outbox o evento `EssaySubmitted` (envelope do §5 da arquitetura, `aggregateVersion = 1`, `correlationId = causationId = commandId`).
6. Gravar a resposta no `command_log` e confirmar a transação. Responder `202`.

7.2 Reserva atômica da cota

```
INSERT INTO write.essay_quota (user_id, period, used)
VALUES ($1, $2, 1)
ON CONFLICT (user_id, period)
DO UPDATE SET used = write.essay_quota.used + 1
WHERE write.essay_quota.used < $3
RETURNING used;
```

`$3` é o limite (`ESSAY_MONTHLY_LIMIT`, hoje 4). A primeira submissão do mês cria a linha com `used = 1`. As seguintes só incrementam se `used < limite`; quando a condição falha, o `UPDATE` não afeta linha e nada é devolvido. Duas submissões simultâneas com 3 usos serializam no bloqueio da linha: a segunda reavalia a condição sobre o valor confirmado e é recusada. Isso é o que torna a regra correta com dois aparelhos ao mesmo tempo.

Devolução da cota (falha permanente, §10.4), na mesma transação que grava a falha:

```
UPDATE write.essay_quota
SET used = used - 1
WHERE user_id = $1 AND period = $2 AND used > 0;
```

7.3 Período da cota e `resetsAt`

O período é calculado no fuso `America/Sao_Paulo` (premissa do §1.6). A função pura vive em `domain/essay/quota-period.ts`. Ela calcula o início do próximo mês confirmando com o `Intl` que o instante candidato cai de fato à meia-noite do dia 1 em São Paulo, em vez de fixar um deslocamento, para tolerar uma eventual mudança de regra de horário.

```
const sp = new Intl.DateTimeFormat('en-CA', {
  timeZone: 'America/Sao_Paulo',
```

```
  year: 'numeric', month: '2-digit', day: '2-digit',
  hour: '2-digit', hourCycle: 'h23',
});

function partsInSaoPaulo(date: Date) {
  const get = (t: string) => Number(sp.formatToParts(date).find((p) => p.type === t)!.value);
  return { year: get('year'), month: get('month'), day: get('day'), hour: get('hour') };
}

export function quotaPeriod(now: Date): string {
  const { year, month } = partsInSaoPaulo(now);
  return `${year}-${String(month).padStart(2, '0')}`;
}

export function startOfNextPeriod(now: Date): Date {
  const { year, month } = partsInSaoPaulo(now);
  const ny = month === 12 ? year + 1 : year;
  const nm = month === 12 ? 1 : month + 1;
  for (const offsetHours of [3, 2, 4]) {
    const candidate = new Date(Date.UTC(ny, nm - 1, 1, offsetHours, 0, 0));
    const p = partsInSaoPaulo(candidate);
    if (p.day === 1 && p.hour === 0) return candidate;
  }
  throw new Error('Início do mês em America/Sao_Paulo não encontrado');
}
```

Testes obrigatórios da função: 31/10/2026 às 23h59 em Brasília ainda é `2026-10`; 01/11/2026 às 00h00 em Brasília já é `2026-11`; dezembro vira janeiro do ano seguinte; `resetsAt` de outubro é `2026-11-01T03:00:00Z`.

7.4 Esqueleto do handler

```
// backend/src/application/commands/submit-essay.handler.ts
import type { CommandAccepted, CommandEnvelope } from '@contracts/commands/command-envelope.ts';
import type { SubmitEssayPayload } from '@contracts/commands/submit-essay.command.ts';
import { quotaPeriod, startOfNextPeriod } from '../../domain/essay/quota-period.ts';
import type { EntitlementPort } from '../ports/entitlement.port.ts';
import type { UnitOfWork } from '../ports/unit-of-work.port.ts';
import { EssayQuotaExceeded, PremiumRequired } from '../errors.ts';

type Deps = {
  uow: UnitOfWork;
  entitlements: EntitlementPort;
  now: () => Date;
  newId: () => string;
  monthlyLimit: number;
};

export function makeSubmitEssayHandler(deps: Deps) {
  return (userId: string, cmd: CommandEnvelope<SubmitEssayPayload>) =>
    deps.uow.run(async (tx): Promise<CommandAccepted> => {
```

```
      const claim = await tx.commandLog.claim(userId, cmd, 'SubmitEssay');
      if (claim.replayed) return claim.response;

      const now = deps.now();
      const plan = await tx.entitlements.get(userId, now);
      if (plan !== 'PREMIUM') throw new PremiumRequired();

      const period = quotaPeriod(now);
      const used = await tx.quota.reserve(userId, period, deps.monthlyLimit);
      if (used === null) {
        throw new EssayQuotaExceeded({
          limit: deps.monthlyLimit,
          used: deps.monthlyLimit,
          resetsAt: startOfNextPeriod(now).toISOString(),
        });
      }

      await tx.essays.insert({ userId, cmd, period });
      await tx.outbox.add({
        eventId: deps.newId(),
        type: 'EssaySubmitted',
        aggregateId: cmd.payload.essayId,
        aggregateVersion: 1,
        correlationId: cmd.commandId,
        causationId: cmd.commandId,
        occurredAt: now.toISOString(),
        payload: { essayId: cmd.payload.essayId, userId },
      });

      const response: CommandAccepted = {
        commandId: cmd.commandId,
        status: 'accepted',
        receivedAt: now.toISOString(),
      };
      await tx.commandLog.complete(userId, cmd.commandId, response);
      return response;
    });
}
```

O `payload` do evento carrega só identificadores (`essayId`, `userId`); o Worker busca o texto no banco. Assim o texto do aluno não viaja pelo Redis nem pelos logs de fila.

8. Outbox transacional e relay

O outbox garante que “gravou” e “vai publicar” aconteçam juntos (D5). O relay é um processo separado (`bootstrap/relay.ts`, um contêiner próprio) que lê `write.outbox` e publica no BullMQ.

8.1 Roteamento por tipo de evento

O outbox guarda o evento uma vez; o relay decide a quais filas ele vai. A tabela abaixo é o contrato de roteamento (ela substitui qualquer decisão local e muda só junto do §5 e do §6 da arquitetura):

Evento	Filas de destino
`QuestionAnswered`, `RoutineProgressRecorded`	`projections`
`EssaySubmitted`	`essay-correction`, `projections`
`EssayCorrected`, `EssayCorrectionFailed`	`projections`, `notifications`
`MockExamCompleted`	`mock-exam-processing`, `projections`
`MockExamProcessed`	`projections`, `notifications`
Mensagem técnica `MessageDeadLettered`	`dead-letter`

A fila `notifications` não está na tabela do §6 da arquitetura: ela só existe se o push (P4) for aprovado e **precisa entrar naquela tabela antes de existir no código**.

8.2 Algoritmo

```
-- Dentro de uma transação por lote
SELECT id, event_type, aggregate_id, correlation_id, envelope
FROM write.outbox
WHERE published_at IS NULL
ORDER BY id
LIMIT 100
FOR UPDATE SKIP LOCKED;
```

Para cada linha, o relay publica um job por fila de destino com `jobId = <type>_<aggregateId>_<correlationId>` e, ao terminar o lote, marca `published_at = now()` e confirma a transação. Se o BullMQ ou o Redis falhar, a transação reverte, `attempts` e `last_error` são atualizados numa transação separada e o lote é tentado de novo depois, com intervalo crescente.

```
// backend/src/workers/relay/outbox-relay.ts (esqueleto)
export async function relayOnce(deps: RelayDeps): Promise<number> {
  return deps.uow.run(async (tx) => {
    const rows = await tx.outbox.claimPending(100); // FOR UPDATE SKIP LOCKED
    for (const row of rows) {
      for (const queue of ROUTES[row.eventType]) {
        await deps.queue.publish(queue, row.eventType, row.envelope, {
          jobId: `${row.eventType}_${row.aggregateId}_${row.correlationId}`,
        });
      }
    }
    await tx.outbox.markPublished(rows.map((r) => r.id));
    return rows.length;
  });
}
```

8.3 Garantias e limites

- **Pelo menos uma vez.** Se o processo cair depois de publicar e antes de confirmar, o lote é republicado. O `jobId` determinístico faz o BullMQ ignorar a duplicata enquanto o job existir, e o consumidor é idempotente pelo `eventId` mesmo depois que o job for removido (§9.4).

- `jobId` **válido.** O BullMQ rejeita ids com `:` e ids só de dígitos; o formato acima usa `_` e começa por letras, então é aceito. Um job removido (por `removeOnComplete`) libera o `jobId`, e por isso o `jobId` não é a garantia de idempotência.
- **Ordem.** Só vale por agregado e é verificada por `aggregateVersion` nas projeções, não pela fila.
- **Redis fora do ar.** A API continua aceitando comandos (o outbox acumula); o relay publica quando o Redis voltar. Alerta: idade da linha pendente mais antiga.
- **Intervalo.** Consulta a cada 500 ms enquanto houver linhas; 2 s quando ocioso (valores iniciais a calibrar). `LISTEN/NOTIFY` é uma otimização futura.
- **Uma instância do relay** basta no início; o `SKIP LOCKED` permite mais de uma sem duplicar o lote.
- **Limpeza.** Linhas publicadas com mais de 7 dias são removidas por uma tarefa periódica (valor a calibrar).

8.4 Reconciliação depois de perda de dados no Redis

A arquitetura promete que, mesmo com perda de dados no Redis, nada se perde (§6 e §11 do documento de arquitetura). O outbox sozinho não cumpre isso: uma linha já marcada como publicada deixa de ser republicada, e o job correspondente some junto com o Redis. A garantia exige um **reconciliador**, executado a cada 5 minutos no processo do relay, que republica o que ficou preso:

```
UPDATE write.outbox o
SET published_at = NULL
FROM write.essays e
WHERE o.event_type = 'EssaySubmitted'
  AND o.aggregate_id = e.id
  AND e.status = 'processing'
  AND o.published_at < now() - interval '15 minutes';
```

A regra é a mesma para `MockExamCompleted` e `write.mock_exams`. O limite de 15 minutos é uma proposta inicial (maior que a soma dos atrasos do backoff do §9.2). A republicação é segura: se o job ainda existir, o `jobId` determinístico faz o BullMQ ignorá-lo; se não existir, ele volta à fila; e o consumidor é idempotente pelo `eventId`. As projeções perdidas são reconstruídas por um script administrativo que relê `write.*` e reaplica os eventos. Esses dois mecanismos são pré-requisito do marco B6 e do runbook de perda do Redis do guia de DevOps.

9. Filas BullMQ e Workers

9.1 Conexões

As exigências abaixo vêm da documentação “Going to production” do BullMQ, consultada em 30/10/2026.

```
// backend/src/infrastructure/queue/connections.ts
import { Redis } from 'ioredis';

// Produtor (API e relay): operações devem falhar rápido se o Redis cair.
export const createProducerConnection = (url: string) =>
  new Redis(url, { enableOfflineQueue: false });

// Worker: maxRetriesPerRequest precisa ser null (exigência do BullMQ).
```

```
export const createWorkerConnection = (url: string) =>
  new Redis(url, { maxRetriesPerRequest: null });
```

9.2 Filas e opções padrão

Filas da arquitetura (§6): `essay-correction`, `mock-exam-processing`, `projections` e `dead-letter` (mais `notifications`, se P4 for aprovada). Nomes em kebab-case.

```
const defaultJobOptions = {
  attempts: 5,
  backoff: { type: 'exponential', delay: 10_000, jitter: 0.5 },
  removeOnComplete: { count: 1_000 },
  removeOnFail: { count: 5_000 },
} as const;
```

No backoff exponencial do BullMQ a espera antes da nova tentativa é `2^(tentativas − 1) × delay`, ou seja, 10, 20, 40 e 80 segundos para as quatro novas tentativas. Esses valores (5 tentativas, 10 s, retenção de 1.000 e 5.000 jobs) são **propostas iniciais** que a arquitetura manda calibrar com teste de carga (§19, B7). Por padrão o BullMQ guarda jobs concluídos e falhos para sempre, e é por isso que a retenção é limitada.

9.3 Workers

```
// backend/src/bootstrap/worker.ts (esqueleto)
const worker = new Worker('essay-correction', processor, {
  connection: createWorkerConnection(env.REDIS_URL),
  concurrency: env.ESSAY_WORKER_CONCURRENCY,           // proposta inicial: 2
  limiter: { max: env.ESSAY_RATE_LIMIT_PER_MIN, duration: 60_000 },
});

async function shutdown() {
  await worker.close();      // termina o job em andamento e minimiza jobs travados
  process.exit(0);
}
process.on('SIGTERM', shutdown);
process.on('SIGINT', shutdown);
process.on('uncaughtException', (e) => logger.error({ err: e }, 'uncaughtException'));
process.on('unhandledRejection', (e) => logger.error({ err: e }, 'unhandledRejection'));
```

- `concurrency` e `limiter` respeitam o limite de taxa e o custo da IA. Ajuste ao limite de taxa da conta no provedor, que precisa ser confirmado (§14 da arquitetura).
- O contêiner precisa de um `stop_grace_period` maior que o maior tempo de um job (tempo limite da IA mais folga), senão o job pode ser interrompido e ficar travado.
- Jobs travados (stalled) são reentregues pelo BullMQ: por isso todo processador é idempotente.

9.4 Idempotência do consumidor

Todo processador começa e termina na mesma regra: o efeito e o registro do `eventId` ocorrem **na mesma transação**.

```
INSERT INTO write.processed_events (consumer, event_id)
VALUES ($1, $2)
ON CONFLICT DO NOTHING
RETURNING event_id;
```

Se nada for devolvido, o evento já foi processado: o processador termina com sucesso sem repetir o efeito. Essa transação é a que grava o resultado (§10). Um job reentregue por travamento, uma duplicata do relay e um reprocesso manual da `dead-letter` ficam, os três, seguros.

9.5 Erros transitórios, permanentes e `dead-letter`

Tipo	Exemplos	Tratamento
Transitório	Tempo limite, `429`, `5xx` da IA; banco indisponível; saída da IA inválida nas primeiras tentativas	Lançar o erro comum; o BullMQ repete com o backoff
Permanente	Payload inválido; conteúdo rejeitado pela IA; saída fora do esquema depois das tentativas	Executar o tratamento de falha final (abaixo) e lançar `UnrecoverableError`
Tentativas esgotadas	A última tentativa também falhou	Mesmo tratamento de falha final

Tratamento de falha final, numa **única transação**: marcar a entidade como `failed` com `failure_code`, devolver a cota (§7.2), inserir no outbox o evento `...Failed` e inserir no outbox a mensagem técnica `MessageDeadLettered` com o contexto do erro (job, tentativas, mensagem), que o relay roteia para a fila `dead-letter`. Só depois disso o processador lança `UnrecoverableError` para que o BullMQ encerre o job como falho sem novas tentativas. Como o tratamento é um passo explícito do código, ele não depende de recurso específico da versão do BullMQ.

Como saber se é a última tentativa: dentro do processador, `job.attemptsMade + 1 >= (job.opts.attempts ?? 1)` (`attemptsMade` conta as tentativas já falhas). Confirme a semântica na versão instalada com um teste.

Reprocesso manual: um comando administrativo interno (fora da API pública, executado por script) relê a `dead-letter`, valida e republica no outbox com `jobId` novo. O guia de DevOps tem o runbook.

9.6 Métricas das filas

O BullMQ 6.5.2 expõe `queue.exportPrometheusMetrics()` (verificado nas definições de tipos), que devolve texto no formato do Prometheus. Métricas mínimas exigidas pela arquitetura (§6): profundidade, idade do job mais antigo, latência de processamento, taxa de falha e crescimento de `dead-letter`. O mais importante é o alerta de crescimento da `dead-letter`.

10. Worker de correção de redação e integração com a IA

10.1 Pipeline

1. Consumir o job `EssaySubmitted`; obter o `eventId` e o `essayId`.
2. Ler (sem registrar nada) se o `eventId` já consta em `processed_events` e carregar a redação; se o evento já foi processado ou a redação já está `completed` ou `failed`, terminar com sucesso.
3. **Fora de transação**, chamar `AiPort.correctEssay` com tempo limite próprio e `AbortSignal` (não segurar conexão do banco durante a chamada à IA).

4. Registrar o uso de tokens em `write.ai_usage` (também em caso de saída inválida, porque a chamada foi cobrada).
5. Validar a saída com `zod`. Inválida: erro transitório; nas tentativas finais, falha permanente (§9.5).
6. Numa transação: gravar `result`, `prompt_version`, `model`, `status = 'completed'`, `version = 2`, e inserir no outbox o `EssayCorrected` (`aggregateVersion = 2`).

O registro em `processed_events` acontece **na transação do passo 6**, junto do resultado: o passo 2 só lê. Se a chamada à IA for refeita por uma reentrega, o resultado gravado e o evento continuam únicos, mas a chamada pode ser cobrada duas vezes; por isso o `ai_usage` registra cada tentativa e as tentativas por saída inválida têm limite baixo (proposta: 2).

10.2 `AiPort`

```
// backend/src/application/ports/ai.port.ts
export type AiUsage = {
  inputTokens: number;
  cachedInputTokens: number;
  outputTokens: number;
};

export type AiCorrection = {
  raw: unknown;       // saída bruta; só o Worker a vê e ela é validada antes de qualquer uso
  model: string;
  usage: AiUsage;
};

export interface AiPort {
  correctEssay(
    input: { essayText: string; promptVersion: string },
    opts: { signal: AbortSignal },
  ): Promise<AiCorrection>;
}

export class TransientAiError extends Error {}  // tempo limite, 429, 5xx: o BullMQ repete
export class PermanentAiError extends Error {}  // conteúdo rejeitado: sem novas tentativas
```

10.3 Adaptador da OpenAI

Fica em `infrastructure/ai/openai.ai.adapter.ts`. Regras:

- O modelo vem de `AI_MODEL`. A página de preços da OpenAI o lista como `gpt-6.1-terra`; confirme o identificador, os limites de contexto e de saída e o formato de saída estruturada na documentação da API **antes** de implementar. Não foi possível ler a página de lançamento do modelo (a requisição falhou), então esses três itens continuam em aberto.
- Construa o cliente com **`maxRetries: 0`**: as tentativas são do BullMQ (backoff, limite de taxa e contagem de falhas). Retentativas escondidas no SDK dobrariam o custo e confundiriam a contagem.
- Tempo limite por chamada em `AI_TIMEOUT_MS`, mais um `AbortSignal` do Worker.
- Traduza erros: tempo limite, `429` e `5xx` viram `TransientAiError`; rejeição de conteúdo e erro `4xx` de requisição viram `PermanentAiError`.
- A chave `OPENAI_API_KEY` existe só no servidor, fora do repositório e dos logs.

- O adaptador **não** loga o texto enviado nem a resposta bruta.
- Envie só o texto da redação e os critérios: nada de `userId`, e-mail ou metadados do aluno (minimização, LGPD).

10.4 Prompts versionados

`infrastructure/ai/prompts/essay-correction.v1.ts` exporta `PROMPT_VERSION` e a função que monta a entrada. O conteúdo do prompt e a rubrica **vêm do produto**: este guia não define critérios nem notas. Duas regras técnicas:

- **Prefixo estável primeiro.** Instruções e rubrica vêm no início e o texto da redação por último. Assim o prefixo pode ser servido do cache de entrada (US$ 0,05 por milhão de tokens, contra US$ 1,00 sem cache).
- **Versão registrada.** Cada resultado grava `prompt_version` e `model`, para auditoria e comparação.

10.5 Custo e freio orçamentário

Cada chamada grava em `write.ai_usage` os tokens (entrada, entrada em cache e saída) e o `cost_usd`, calculado com os preços configurados (`AI_PRICE_INPUT_PER_M=1.00`, `AI_PRICE_CACHED_INPUT_PER_M=0.05`, `AI_PRICE_OUTPUT_PER_M=5.00`). A fórmula é:

`cost_usd = (entrada_sem_cache × preço_entrada + entrada_em_cache × preço_cache + saída × preço_saída) ÷ 1.000.000`

Com esses registros, as estimativas do §1.8 (US$ 34 por mês a cada 1.000 assinantes Premium no cenário base) são substituídas por medições logo nas primeiras semanas. Opcionalmente, `AI_MONTHLY_BUDGET_USD` ativa um freio: antes de cada chamada o Worker compara o gasto do mês e, ao ultrapassar o valor, pausa a fila (`worker.pause()`) e dispara um alerta. As redações continuam na fila, não falham. O freio é recomendado e entra no marco B6 se aprovado.

10.6 O que o Worker nunca faz

Não devolve a saída bruta da IA ao app, não expõe detalhes de erro do provedor em nenhuma resposta, não mantém conexão de banco aberta durante a chamada à IA e não envia dado além do necessário ao provedor.

11. Projeções e lado de leitura

11.1 Projetores

Projetores (`application/events`) consomem a fila `projections` e atualizam `read.*`. Há duas regras de idempotência, escolhidas pelo tipo de projeção:

- **Linha por agregado** (`essay_view`, `mock_exam_view`): aplicar somente se `aggregateVersion` do evento for maior que a da linha. Um evento antigo ou repetido é ignorado.

```
INSERT INTO read.essay_view
  (essay_id, user_id, status, aggregate_version, created_at, as_of)
VALUES ($1, $2, $3, $4, $5, now())
ON CONFLICT (essay_id) DO UPDATE
SET status = EXCLUDED.status, result = $6, failure_code = $7,
    aggregate_version = EXCLUDED.aggregate_version, as_of = now()
WHERE read.essay_view.aggregate_version < EXCLUDED.aggregate_version;
```

- **Acumuladores** (`performance_summary`): registrar `(consumer, event_id)` em `write.processed_events` na mesma transação do incremento.

O `as_of` de cada linha é o instante da aplicação; é ele que a API devolve.

11.2 Consultas

Handlers de consulta (`application/queries`) leem **apenas** `read.*`, não importam `domain/` e sempre devolvem `asOf`. Cada consulta filtra por `user_id` do token: um usuário nunca lê dado de outro.

- `GetEssay`: devolve `404` quando a linha não existe. Logo depois de um `202`, a projeção ainda pode não ter sido aplicada; o app sabe disso e trata o `404` transitório como "em processamento" (guia do Front-End).
- `GetEssayList` (consulta proposta, precisa de aprovação): lista as redações do usuário em `read.essay_view`, paginada por `created_at`.
- `GetPerformanceSummary`, `GetMockExamHistory` e `GetMockExamResult`: leitura direta das projeções, com os campos definidos com o produto.

11.3 Consistência e atraso

Se o projetor atrasa, a consulta devolve um `asOf` antigo e a interface sinaliza o atraso. O alerta correspondente (idade do job mais antigo da fila `projections`) está no guia de DevOps.

12. API HTTP

12.1 Rotas

Método e rota	Tipo	Observação
`POST /v1/auth/login`, `/v1/auth/refresh`, `/v1/auth/logout`	Autenticação	HTTP síncrono, não são comandos (§13)
`POST /v1/commands/answer-question`	Comando	`202`
`POST /v1/commands/record-routine-progress`	Comando	`202`
`POST /v1/commands/submit-essay`	Comando	`202` ou `403` (Premium e cota)
`POST /v1/commands/complete-mock-exam`	Comando	`202`
`POST /v1/commands/register-push-token`	Comando técnico	Só se P4 for aprovada
`GET /v1/essays/{essayId}`, `/v1/me/essays`	Consulta	`/v1/me/essays` é proposta
`GET /v1/me/entitlements`, `/v1/me/performance`, `/v1/me/mock-exams`, `/v1/mock-exams/{id}`	Consulta	Com `asOf`
`GET /healthz`	Operação	Liveness: o processo responde
`GET /readyz`	Operação	Readiness: Postgres e Redis respondem
`GET /metrics`	Operação	Porta interna; **nunca** exposta pelo proxy

As rotas de comando e as de consulta ficam em arquivos separados (`api/routes/commands` e `api/routes/queries`), como pede a arquitetura. As rotas chamam o handler diretamente; não há barramento genérico.

12.2 Validação na borda

Todo corpo, parâmetro e cabeçalho é validado com `zod` antes do handler. O esquema do backend precisa ser **compatível com o tipo de `contracts/`** em tempo de compilação, com `satisfies`, o que evita divergência sem exigir `zod` em `contracts/` (P6):

```
import { z } from 'zod';
import type { SubmitEssayPayload } from '@contracts/commands/submit-essay.command.ts';

export const submitEssayPayloadSchema = (maxChars: number) =>
  z.object({
    essayId: z.uuid(),
    text: z.string().min(1).max(maxChars),
  }) satisfies z.ZodType<SubmitEssayPayload>;
```

O limite `maxChars` vem da configuração (`ESSAY_MAX_CHARS`), definido pelo produto e **sem valor padrão inventado**. O `commandId` precisa ser UUID v4 e o `issuedAt` uma data ISO 8601; um relógio do aparelho absurdo (por exemplo anos no futuro) é rejeitado com `VALIDATION_FAILED`, e a decisão de tolerância é do produto.

12.3 Erros

Um único tratador de erros traduz erros de domínio para o `ApiError` do §1.7 (mesmo corpo, `correlationId` presente). Erro inesperado devolve `INTERNAL_ERROR` com mensagem genérica e vai para os logs e para o Sentry com o `correlationId`, nunca com o corpo da requisição.

Erro de domínio	Resposta
`PremiumRequired`	`403` `PREMIUM_REQUIRED`
`EssayQuotaExceeded`	`403` `ESSAY_QUOTA_EXCEEDED` com `limit`, `used`, `resetsAt`
`CommandIdConflict`	`409` `COMMAND_ID_CONFLICT`
Falha de validação	`400` `VALIDATION_FAILED`
Token ausente ou inválido	`401` `UNAUTHENTICATED`
Excesso de requisições	`429` `RATE_LIMITED` com `Retry-After`

12.4 Segurança HTTP

- `@fastify/helmet` com os cabeçalhos padrão; `@fastify/cors` **desabilitado** (o app nativo não precisa de CORS); habilitar só se a web for adotada, com lista explícita de origens.
- `@fastify/rate-limit` global e mais restrito em `/v1/auth/*`. Chave de limite: `userId` quando autenticado, IP caso contrário.
- `trustProxy` ligado, porque a API fica atrás do Caddy; o IP real vem de `X-Forwarded-For` somente do proxy.
- `bodyLimit` baixo para comandos (valor a calibrar com o limite do texto da redação) e tempo limite de requisição.

- O servidor escuta em `0.0.0.0` apenas dentro da rede do Compose; a única porta pública é a do proxy (guia de DevOps).

13. Autenticação e identidade

Proposta P1: login com Google e com Apple (OIDC) sem senha. Ela não exige envio de e-mail, não guarda senha e não custa nada. Se o produto preferir outro método, só `/v1/auth/*` e o §13 mudam.

13.1 `POST /v1/auth/login`

Recebe `{ provider, idToken }`. A API:

1. Valida o `idToken` com `jose` contra as chaves públicas do provedor (`createRemoteJWKSet`), conferindo assinatura, `iss`, `aud` (os identificadores de cliente do app: `GOOGLE_CLIENT_IDS` e `APPLE_CLIENT_ID`), `exp` e, quando usado, o `nonce`. Os endereços de chaves e os valores de `iss` dos provedores devem ser reconfirmados nas documentações do Google e da Apple na implementação.
2. Encontra ou cria `write.users` por `(provider, provider_sub)`.
3. Emite um JWT de acesso e um refresh token opaco aleatório.
4. Responde `{ accessToken, accessTokenExpiresAt, refreshToken }`.

13.2 Tokens

- **Acesso:** JWT HS256 assinado com `JWT_SECRET` (pelo menos 32 bytes aleatórios), com `sub = userId`, `iss`, `aud`, `iat` e `exp`. Validade curta (proposta inicial: 15 minutos). Só a API o verifica, então a chave simétrica basta.
- **Refresh:** 32 bytes aleatórios codificados. O banco guarda apenas o SHA-256 (`token_hash`), nunca o token. Validade inicial proposta de 30 dias.
- **Rotação:** cada renovação (`/v1/auth/refresh`) revoga o token usado e emite outro da mesma `family_id`. **Se um token já revogado for apresentado, toda a família é revogada** (indício de roubo) e o usuário precisa entrar de novo.
- **Logout:** revoga a família do token apresentado.
- Em todas as rotas autenticadas, um hook valida o JWT e coloca `userId` no contexto; o handler nunca confia em `userId` vindo do corpo.

13.3 Dados pessoais

O backend guarda apenas `provider`, `provider_sub` e, nas demais tabelas, `user_id`. Não guarda e-mail nem nome. A exclusão da conta e a retenção dependem da decisão de LGPD (em aberto) e não são implementadas sem ela.

14. Plano Premium: `EntitlementPort`

```
// backend/src/application/ports/entitlement.port.ts
export type Plan = 'FREE' | 'PREMIUM';

export interface EntitlementPort {
  /** Plano vigente do usuário no instante `at`; PREMIUM só se estiver dentro da validade. */
  get(userId: string, at: Date): Promise<Plan>;
}
```

O adaptador lê `write.entitlements` (`plan = 'PREMIUM'` e `valid_until` nulo ou posterior a `at`); sem linha, é `FREE`. **A fonte que preenche essa tabela é decisão em aberto** (§1.6), e o custo de cada caminho está no §1.8. Nenhum endpoint público concede Premium.

Exceção proposta: `GetEntitlements` lê do write store. A consulta mostra o plano e quantas correções restam. Esse dado é pequeno, indexado por usuário e precisa ser exato (o app não pode dizer "restam 2" quando restou 1). Passá-lo por uma projeção exigiria novos eventos (`PlanChanged`, `UserRegistered`) fora do catálogo. A proposta é um handler de consulta que lê `write.entitlements` e `write.essay_quota` **pela `EntitlementPort`** (a mesma porta do comando), documentado como exceção à regra "consultas só leem o read store". Isso precisa de aprovação e está no §14 do documento de arquitetura; a alternativa é criar os eventos e a projeção.

Resposta (`GetEntitlements`): `plan`, `essayQuota` (`limit`, `used` do período atual, `resetsAt`) ou `null` quando o plano não dá direito, e `asOf` igual ao instante da leitura.

15. Observabilidade

15.1 Logs

`pino` em JSON no `stdout` (o Docker coleta). Todo log e todo job carrega `correlationId` (o `commandId`). Campos comuns: `level`, `time`, `msg`, `correlationId`, `route` ou `queue`, `jobId`, `eventId`. O `redact` do `pino` remove `req.headers.authorization`, `*.text`, `*.idToken`, `*.refreshToken` e `*.accessToken`. **Texto de redação, tokens e chaves nunca aparecem em log.**

15.2 Métricas

Servidas em porta interna por `prom-client` mais `queue.exportPrometheusMetrics()`:

Métrica	Uso
`http_request_duration_seconds` (por rota e status)	Latência e erros da API
`outbox_pending` e `outbox_oldest_pending_age_seconds`	Relay parado ou Redis fora
Profundidade, idade do job mais antigo, latência e falhas por fila	Atraso de Workers e projeções
`dead_letter_total`	Crescimento da `dead-letter` (alerta principal)
`ai_requests_total{outcome}` e `ai_tokens_total{kind}`	Uso e falhas da IA
`ai_cost_usd_total`	Custo acumulado
`essay_quota_rejections_total{reason}`	Rejeições por Premium e por cota

15.3 Erros

`@sentry/node` (plano gratuito, 5 mil erros por mês) com `sendDefaultPii: false` e filtro que remove o corpo das requisições. Alertas e painéis estão no guia de DevOps.

16. Segurança e LGPD

- **Segredos fora do repositório** (público): `OPENAI_API_KEY`, `JWT_SECRET`, senhas de banco e Redis vêm de variáveis de ambiente entregues pelo servidor; `.env` entra no `.gitignore` antes do primeiro uso.

- **Papéis de banco com menor privilégio:** um papel para a API e o relay (gravação em `write`, leitura de `read`), outro para Workers (gravação em `write` e `read`) e outro só para migrações. O papel de consultas lê somente `read` (e `write.entitlements` e `write.essay_quota`, pela exceção do §14).
- **Validação na borda** de comando, consulta, evento e saída da IA.
- **Autorização por objeto:** toda consulta e todo comando filtram por `userId` do token; trocar `essayId` na rota não pode expor a redação de outro usuário (teste obrigatório).
- **Dados em trânsito:** TLS pelo Caddy; Postgres e Redis só na rede interna, com senha. Redis com `requirepass`.
- **Minimização:** o texto do aluno só trafega do servidor para o provedor de IA, sem identificadores; o Redis e os logs nunca recebem o texto.
- **LGPD em aberto** (§14 da arquitetura): base legal, retenção, exclusão e consentimento para envio ao provedor. Nenhum comando ou consulta que dependa deles é implementado sem a decisão; não crie rotina de exclusão por conta própria.
- **Dependências:** `npm audit` e atualizações regulares; novas dependências só com aprovação.
- **Reprocesso de `dead-letter`:** feito por script administrativo, nunca por rota pública.

17. Testes

Executor proposto: Vitest (P7). Os testes de integração leem `DATABASE_URL` e `REDIS_URL` do ambiente; localmente vêm do Docker Compose de desenvolvimento e, na integração contínua, de contêineres de serviço (guia de DevOps).

Alvo	O que verificar
`quota-period`	Os quatro casos do §7.3, incluindo a virada em Brasília
Idempotência	Reenvio devolve o mesmo `202`; payload diferente com o mesmo `commandId` dá `409`; reenvio concorrente cria um único efeito
`SubmitEssay`	Sem Premium (`403`); com 0, 3 e 4 usos; duas submissões simultâneas com 3 usos (uma passa); reenvio não consome a cota de novo; perda do Premium depois da aceitação não invalida o reenvio
Devolução da cota	Falha permanente devolve 1 e o contador nunca fica negativo
Outbox e relay	Linha gravada com a transação; relay publica uma vez por fila de destino; queda após publicar gera duplicata absorvida; reconciliador republica uma redação `processing` cujo job sumiu do Redis
BullMQ (integração)	`jobId` duplicado ignorado; retry com backoff; `UnrecoverableError` encerra sem repetir; `dead-letter` recebe a mensagem; processador idempotente por `eventId`
Worker de correção	Com um `AiPort` falso e determinístico (um dublê de teste, não dado de produto): sucesso, erro transitório seguido de sucesso, saída inválida, falha permanente
Projeções	Evento antigo ignorado; evento repetido não duplica contador; `asOf` atualizado
API	Cada status e `code` do §1.7; `404` de `GetEssay`; um usuário não lê a redação de outro; tokens inválidos e expirados; limite de taxa
Autenticação	`idToken` com `aud`, `iss` ou `exp` errado é recusado; rotação de refresh; reuso de token revogado revoga a família

Alvo	O que verificar
Contrato	O esquema `zod` de cada comando é atribuível ao tipo de `contracts/` (falha de `tsc` se divergir)
Ponta a ponta do §8.1	Premium envia redação, `202`, relay, Worker (IA falsa), projeção e `GetEssay` com `completed`

Toda nova fronteira (comando, consulta, evento, saída da IA) é testada em sucesso, entrada inválida, ausência e falha. O teste de carga (B7) é descrito no guia de DevOps; o backend fornece os cenários: rajada de comandos (latência do `202`), drenagem de fila com IA falsa de latência fixa e concorrência de cota. As metas iniciais (por exemplo, `p95` do `202` abaixo de 300 ms) são propostas a confirmar com a equipe.

18. Variáveis de ambiente

Somente nomes: valores nunca entram no repositório, nos logs nem nesta documentação.

Variável	Uso	Observação
`NODE_ENV`, `PORT`, `LOG_LEVEL`	Execução	`PORT` da API
`DATABASE_URL`	PostgreSQL	Um papel por processo
`REDIS_URL`	BullMQ	Com senha
`JWT_SECRET`, `JWT_ISSUER`, `JWT_AUDIENCE`	Tokens de acesso	Segredo com pelo menos 32 bytes
`GOOGLE_CLIENT_IDS`, `APPLE_CLIENT_ID`	Validação do `idToken`	Valores públicos
`OPENAI_API_KEY`	IA	**Segredo**; só no servidor e apenas no ambiente do Worker
`AI_MODEL`, `AI_TIMEOUT_MS`	IA	Modelo confirmado na documentação da API
`AI_PRICE_INPUT_PER_M`, `AI_PRICE_CACHED_INPUT_PER_M`, `AI_PRICE_OUTPUT_PER_M`	Cálculo de custo	1,00 / 0,05 / 5,00 em 30/10/2026
`AI_MONTHLY_BUDGET_USD`	Freio orçamentário	Opcional
`ESSAY_MONTHLY_LIMIT`	Cota por mês	4 (D9)
`ESSAY_MAX_CHARS`	Tamanho do texto	**Definido pelo produto; sem padrão**
`ESSAY_WORKER_CONCURRENCY`, `ESSAY_RATE_LIMIT_PER_MIN`	Worker	Propostas iniciais a calibrar
`METRICS_PORT`	Métricas	Porta interna
`SENTRY_DSN`	Erros	Público por natureza

19. Ordem de implementação e critérios de aceite

Os marcos `F#` e `D#` referem-se aos guias do Front-End e do DevOps.

Marco	Escopo	Critério observável	Depende de
B0	Fundação: aprovação de P2, P3, P6, P7 e P8; `backend/package.json`, `tsconfig.json`, `lint`, `typecheck`, `test`; Compose de desenvolvimento (Postgres e Redis)	`npm run typecheck`, `lint` e `test` passam; um arquivo `.ts` executa no Node 24 com o alias para `contracts/`	—
B1	Persistência: esquemas `write` e `read`, migrações, papéis, `command_log`, outbox	Migração aplica e reverte em banco limpo; idempotência testada com concorrência	B0
B2	API e autenticação: Fastify, modelo de erro, login Google e Apple, refresh rotativo, `GetEntitlements`	Casos do §17 de API e autenticação passam; login real em aparelho com a API de desenvolvimento	B1, D1
B3	Comandos rápidos, relay, BullMQ e projeções (`AnswerQuestion`, `RecordRoutineProgress`)	Comando gravado, publicado e projetado uma única vez, mesmo com Redis derrubado durante o envio	B2
B4	Redação: `SubmitEssay` com Premium e cota, Worker, `AiPort`, `ai_usage`, `dead-letter`	Todos os testes do §7 e §10 passam; ponta a ponta com IA falsa; uma chamada real à IA, com o prompt e a rubrica do produto, verificada e registrada	B3 e rubrica
B5	Simulados: `CompleteMockExam`, Worker de simulados	Fluxo do §8.2 da arquitetura verificado	B4 e regras de simulado
B6	Observabilidade e endurecimento: métricas, alertas, Sentry, freio orçamentário (se aprovado), papéis de banco, rate limit	Painéis e alertas ativos em D5; falha forçada de IA gera alerta	B4, D5
B7	Carga e prontidão	Testes de carga do §17 executados na VM, valores de tentativas, retenção e concorrência calibrados e registrados	B6, D3

20. Checklist de entrega e pendências

20.1 Definição de pronto de uma tarefa de back-end

- Altera apenas os arquivos necessários e respeita as regras do §4.1; sem barramento genérico nem abstração de uso único.
- Fronteira nova testada em sucesso, inválido, ausente e falha; `typecheck`, `lint` e `test` executados com os resultados informados.
- Sem segredo, sem texto de redação nem token em log; sem dado inventado.
- Contrato novo aprovado e adicionado a `contracts/` como tipo; esquema `zod` com `satisfies`.
- Documentos alinhados e PDFs regenerados quando houve mudança de decisão; commit no padrão `(tipo): Frase no passado`, sem mencionar ferramentas de assistência.

20.2 Pendências que bloqueiam trabalho

Pendência	Quem decide	Bloqueia
Aprovação de P1, P2, P3, P6, P7, P8 e P9	Responsável pelo produto	B0 em diante
Exceção de `GetEntitlements` (leitura do write store)	Arquiteto responsável	B2
Fonte do plano Premium e preenchimento de `write.entitlements`	Produto	Premium real em produção
Rubrica, formato do `result` e prompt da correção	Produto	B4
Identificador, limites de contexto e de saída e formato estruturado do GPT 6.1 - Terra; limite de taxa da conta	Equipe do backend, na documentação da API	B4
Campos de negócio dos comandos, `ESSAY_MAX_CHARS` e regras de simulado	Produto	B3 a B5
Fila `notifications`, comando `RegisterPushToken` e consulta `GetEssayList`	Aprovação	Push e histórico
LGPD: base legal, retenção, exclusão e consentimento	Produto e jurídico	Dados pessoais em produção
Janela da cota (mês corrido) e política de tolerância do relógio	Produto	B4